What happens when your system receives a message it cannot process... and keeps failing again and again? Does it crash... retry forever... or silently lose data?

# What is a Dead Letter Queue (DLQ)?

- A separate queue for failed messages
- Stores messages that cannot be processed successfully
- Prevents system blocking or crashes

# Why DLQ is Needed?

- Prevents infinite retry loops
- Avoids blocking main processing pipeline
- Helps isolate problematic data
- Improves system stability

# How DLQ Works?

- Message sent to main queue
- Consumer tries to process it
- If failure happens → message is retried
- After max retries → sent to DLQ

# What Goes into DLQ?

- Invalid data messages
- Schema mismatch events
- Processing failures (runtime errors)
- External service failures

# Why DLQ is Important in Production?

- Ensures no message is lost
- Helps debugging failed cases
- Improves system reliability
- Keeps main system clean and fast

# How Developers Handle DLQ Messages?

- Log and monitor failed messages
- Fix root cause and reprocess
- Create alerting system for DLQ growth
- Manual or automated reprocessing

# Common Mistakes

- Ignoring DLQ messages
- Not setting retry limits
- No monitoring on failed queue
- Treating DLQ as “ignore zone”

# Real-World Use Cases